FULL PRACTICE SET — Encapsulation, Getters, Setters, OOP

Example 1 — Public / Protected / Private

```
class Sample:
    def __init__(self):
        self.a = 10       # public
        self._b = 20      # protected
        self.__c = 30     # private
```

Example 2 — Name Mangling

```
class Test:
    def __init__(self):
        self.__x = 99

obj = Test()
print(obj._Test__x)    # Not recommended
```

Example 3 — Getter & Setter

```
class Person:
    def __init__(self, age):
        self.__age = age

    @property
    def age(self):
        return f"{self.__age} years old"

    @age.setter
    def age(self, new_age):
        if new_age < 0:
            raise Exception("Invalid age")
        self.__age = new_age
```

MAIN PRACTICE QUESTIONS

1. Student Class Question

Create class **Student** with:

- Public: `name`
- Protected: `_roll_no`
- Private: `__marks` Use getter + setter with marks validation (0–100)

Starter Code

```
class Student:
    def __init__(self, name, roll_no, marks):
        self.name = name
        self._roll_no = roll_no
        self.__marks = marks

    @property
    def marks(self):
        return self.__marks

    @marks.setter
    def marks(self, new_marks):
        # TODO
        pass

    def show_info(self):
        print(self.name, self._roll_no, self.__marks)
```

2. Employee Class Question

Create class **Employee** with:

- Private salary
- Setter must increase salary only with positive amount
- Getter must return `"Rs. <salary>"`

Starter Code

```
class Employee:
    def __init__(self, name, department, salary):
        self.name = name
        self._department = department
```

```
        self.__salary = salary

    @property
    def salary(self):
        # TODO
        pass

    @salary.setter
    def salary(self, amount):
        # TODO
        pass
```

---

3. Book Class Question

Private attribute: `__price` Setter must prevent negative price

Starter Code

```
class Book:
    def __init__(self, title, author, price):
        self.title = title
        self._author = author
        self.__price = price

    @property
    def price(self):
        return self.__price

    @price.setter
    def price(self, new_price):
        # TODO
        pass
```

---

4. Laptop Class Question

Private `__price` Setter: value must be > 20,000 and must be number Getter returns `"Rs. <price>"`

Starter Code

```
class Laptop:
    def __init__(self, brand, ram, price):
        self.brand = brand
        self._ram = ram
        self.__price = price
```

```
    @property
    def price(self):
        # TODO
        pass

    @price.setter
    def price(self, new_price):
        # TODO
        pass
```

5. MobilePhone Class Question

Private `__storage` Setter increases storage **only if positive**

Starter Code

```
class MobilePhone:
    def __init__(self, model, battery, storage):
        self.model = model
        self._battery = battery
        self.__storage = storage

    @property
    def storage(self):
        return self.__storage

    @storage.setter
    def storage(self, extra):
        # TODO
        pass
```

6. Movie Class Question

Private `__rating` Rating must be 1–10 Getter should return `"8 ☆"` format

Starter Code

```
class Movie:
    def __init__(self, name, director, rating):
        self.name = name
        self._director = director
        self.__rating = rating
```

```
    @property
    def rating(self):
        # TODO
        pass

    @rating.setter
    def rating(self, new_rating):
        # TODO
        pass
```

---

7. Bike Class — Write your own

write full class yourelf.

EXTRA PRACTICE QUESTIONS (no starter code)

8. Create class **BankUser** with private balance and PIN.
9. Create class **Car** with getter for model and setter for price.
10. Create class **Product** that has discount logic in setter.

FINAL CLEAN CALCULATOR EXAMPLE (Best OOP Version)

✔ All logic stays inside the class

✔ Main program is extremely clean

✔ see why classes make code easier

Calculator Class

```
# calculator.py
class Calculator:
    def show_operations(self):
        print("\n=== SIMPLE CALCULATOR ===")
        print("1. Add")
        print("2. Subtract")
        print("3. Multiply")
        print("4. Divide")
        print("5. Exit")
```

```
    def get_numbers(self):
        try:
            a = float(input("Enter first number: "))
            b = float(input("Enter second number: "))
            return a, b
        except ValueError:
            print("Invalid numbers!")
            return None, None

    def add(self, a, b): return a + b
    def sub(self, a, b): return a - b
    def mul(self, a, b): return a * b
    def div(self, a, b): return "Error (divide by zero)" if b == 0 else a /
b

    def run(self):
        while True:
            self.show_operations()
            choice = input("Choose (1-5): ")

            if choice == "5":
                print("Goodbye!")
                break

            a, b = self.get_numbers()
            if a is None:
                continue

            if choice == "1": print("Result:", self.add(a, b))
            elif choice == "2": print("Result:", self.sub(a, b))
            elif choice == "3": print("Result:", self.mul(a, b))
            elif choice == "4": print("Result:", self.div(a, b))
            else:
                print("Invalid option!")
```

Main Program

```
# main.py
from calculator import Calculator
calc = Calculator()
calc.run()
```

Now run your code

```
python3 main.py
```